

Blog & App Deployment: Distill, Quatro, Shiny

Agenda

- Blog Deployment: Distill (quick intro) → Quarto (deep dive)
 - Step-by-step: create, customize, and publish your first Quarto blog
- Shiny App Deployment
 - shinyapps.io, Posit Connect, and GitHub Pages options

Websites & Blogs with Distill

What is Distill?

- The `Distill` package creates beautiful, scientific-style blogs and websites directly from R Markdown
- It was the go-to tool for R-based blogs for several years — simple, clean output, easy GitHub Pages deployment
- Still totally functional, but **Quarto has now superseded it** for most use cases

Quick Distill Setup (for context)

```
install.packages("distill")
```

- File → New Project → New Directory → Distill Blog
- Check "Configure for GitHub Pages" — outputs to a [docs/](#) folder instead of [_site/](#)
- Build the site via the **Build** pane → **Build Website**
- Push to GitHub, then enable Pages from [docs/](#) in repo Settings

Key Distill concepts you'll recognize in Quarto:

- `_site.yml` → controls navigation, base URL, theme
- `distill::create_post()` → scaffolds a new post
- `output_dir: "docs"` → GitHub Pages compatibility
- `draft: true` → hides posts from the built site

Why Move to Quarto?

Distill vs. Quarto

Feature	Distill	Quarto
Language support	R only	R, Python, Julia, Observable
Theming	Limited CSS	Full Bootstrap + SCSS
Blog features	Basic	Listings, cards, search, RSS
Extensions	None	Growing ecosystem
Active development	Maintenance mode	Actively developed by Posit
Output formats	HTML	HTML, PDF, Word, slides, books

Bottom line: Quarto does everything Distill does, plus much more.

If you're starting fresh, **start with Quarto.**

Your First Quarto Blog

A Step-by-Step Guide

Step 0: Prerequisites

Before you begin, make sure you have:

- **Quarto CLI** installed: quarto.org/docs/get-started
 - In the RStudio Terminal: `quarto --version` to verify
- **RStudio 2022.07+** (recommended) — has native Quarto support
- A **GitHub account** and Git configured locally

```
# Check Quarto version from R
quarto::quarto_version()

# Install quarto R package if needed
install.packages("quarto")
```

Step 1: Create the Quarto Blog Project

Option A — RStudio GUI (recommended)

File → New Project → New Directory → **Quarto Blog**

Fill in:

- Directory name (e.g., `my-data-blog`)
- Choose your subdirectory
- ☒ Check **"Create a git repository"**
- ☒ Check **"Use visual markdown editor"** (optional but helpful)

Step 1: Create the Quarto Blog Project

Option B — Terminal

```
quarto create project blog my-data-blog  
cd my-data-blog
```

Either way, RStudio opens the new project automatically.

Step 1b: What Gets Created?

```
my-data-blog/
├── _quarto.yml          # Site-wide configuration
├── index.qmd            # Blog listing page (home)
├── about.qmd           # About page
├── posts/              # All your blog posts live here
│   ├── welcome/
│   │   └── index.qmd    # Sample post
│   └── post-with-code/
│       └── index.qmd    # Sample post with R code
├── styles.css           # Custom CSS (start here for styling)
└── _freeze/            # Cached computation outputs
```

Open [_quarto.yml](#) — this is your command center.

Step 2: Understand _quarto.yml

This file controls your entire site:

```
project:
  type: website
  output-dir: docs          # ← GitHub Pages serves from here

website:
  title: "My Data Blog"
  navbar:
    right:
      - about.qmd
      - icon: github
        href: https://github.com/YOUR_USERNAME
      - icon: rss
        href: index.xml
```

output-dir: docs is what makes GitHub Pages deployment work.

Step 2: Understand _quarto.yml

```
format:  
  html:  
    theme: cosmo           # ← Bootswatch theme  
    css: styles.css  
    toc: true
```

Step 2b: Choosing a Theme

Quarto uses [Bootstrap](#) themes — 25+ options, all free:

```
format:
  html:
    theme: cosmo      # clean, modern (default)
    # theme: flatly   # flat, minimal
    # theme: litera    # nice typography
    # theme: lux       # elegant
    # theme: darkly    # dark mode
    # theme: minty     # soft greens
```

You can also use **light + dark** themes together:

```
format:
  html:
    theme:
      light: flatly
      dark: darkly
```


Step 3: Configure the Blog Listing

Open `index.qmd` — it controls your home page:

```
---
title: "My Data Science Blog"
listing:
  contents: posts           # looks in the /posts folder
  sort: "date desc"        # newest first
  type: default            # try: grid, table
  categories: true         # ← adds category sidebar
  sort-ui: false
  filter-ui: false
page-layout: full
title-block-banner: true
---
```

Listing types:

- `type: default` — title, description, date, image in rows

Step 4: Write Your First Post

Each post lives in its own folder inside `posts/`:

```
posts/
├── my-first-post/
│   ├── index.qmd      # the post itself
│   └── image.png      # thumbnail (optional but nice!)
```

Create a new post folder in the Terminal:

```
# From your project root:
mkdir posts/my-first-analysis
touch posts/my-first-analysis/index.qmd
```

Step 4b: Post YAML Front Matter

Every post starts with a YAML header:

```
---
title: "Temperature and School Discipline: First Look"
description: "Exploring the relationship between heat and office"
author: "Maithreyi Gopalan"
date: "2026-02-18"
categories: [education policy, data visualization, R]
image: "thumbnail.png"           # shows on the listing page
draft: false                     # set true to hide while working
bibliography: references.bib     # optional: citation support
---
```

Then write your post in Quarto markdown below the `---`:

Introduction

This post explores...

Step 4c: Quarto Chunk Options

In Quarto, chunk options go **inside** the chunk with `#|` prefix:

You can still use the old `{r echo=FALSE}` style, but `#|` is preferred in Quarto — it works across R, Python, and Julia.

Global defaults go in `_quarto.yml`:

```
execute:
  echo: true
  warning: false
  message: false
  freeze: auto      # ← only re-runs when source changes
```

Step 5: Preview Your Blog Locally

Option A — RStudio

Click the **Render** button on any `.qmd` file, or use **Build** pane → **Render Website**

Option B — Terminal (recommended)

```
quarto preview
```

This starts a local server at <http://localhost:4200> and **live-reloads** as you edit.

```
quarto render                                # builds entire site
quarto render posts/my-first-post/index.qmd  # renders one page
```

The `freeze` folder

Step 6: Push to GitHub

6a — Create a GitHub Repository

1. Go to github.com → New repository
2. Name it (e.g., [my-data-blog](#)) — do not initialize with README
3. Copy the remote URL

6b — Connect your local project

In the RStudio Terminal:

```
git init                                # if not already a repo
git add .
git commit -m "Initial Quarto blog"
git remote add origin https://github.com/YOUR_USERNAME/my-data-b
git branch -M main
git push -u origin main
```

Or use RStudio's **Git pane**: Stage all → Commit → Push    22 / 48

Step 7: Enable GitHub Pages

1. Go to your repo on GitHub
2. Click **Settings** → **Pages** (left sidebar)
3. Under **Source**: select **Deploy from a branch**
4. Branch: `main` | Folder: `/docs`
5. Click **Save**

GitHub will give you a URL like:

`https://YOUR_USERNAME.github.io/my-data-blog/`

It may take 1–2 minutes to go live. 🎉

Important: Make sure `output-dir: docs` is in `_quarto.yml` AND the `docs/` folder is committed to GitHub (it should **not** be in `.gitignore`).

Step 7b: Update Your Base URL

Once you know your site URL, add it to `_quarto.yml`:

```
website:  
  title: "My Data Blog"  
  site-url: https://YOUR_USERNAME.github.io/my-data-blog/  
  description: "Data science for education policy"
```

This enables:

- Correct Open Graph / social sharing cards
- Proper RSS feed links
- Citation metadata

Step 8: Your Ongoing Deployment Workflow

Every time you write a new post or make changes:

```
# 1. Write or edit your .qmd files

# 2. Render the whole site
quarto render

# 3. Stage, commit, and push
git add .
git commit -m "Add post on temperature and discipline"
git push
```

GitHub Pages automatically updates within ~1 minute of your push.

Pro tip: Add to [.gitignore](#) to skip large temporary files:

`/.quarto/`

Optional: GitHub Actions for Auto-Rendering

Instead of rendering locally and pushing [docs/](#), let GitHub do it for you on every push.

Create [.github/workflows/publish.yml](#):

```
on:
  push:
    branches: main

jobs:
  build-deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: quarto-dev/quarto-actions/setup@v2
      - uses: r-lib/actions/setup-r@v2
      - uses: r-lib/actions/setup-r-dependencies@v2
      - uses: quarto-dev/quarto-actions/publish@v2
        with:
          target: gh-pages
```

Customizing: Navigation

Edit `_quarto.yml` to add pages and social links:

```
website:
  title: "My Data Blog"
  navbar:
    left:
      - text: "Posts"
        href: index.html
      - text: "Research"
        href: research.qmd
      - text: "Teaching"
        href: teaching.qmd
    right:
      - icon: github
        href: https://github.com/YOUR_USERNAME
      - icon: twitter
        href: https://twitter.com/YOUR_HANDLE
      - icon: rss
        href: index.xml
```

Icons from Bootstrap Icons — use the icon name directly.

Customizing: Categories

Add `categories:` to any post's YAML:

```
categories: [education policy, R, data visualization, econometric]
```

Enable the category sidebar in `index.qmd`:

```
listing:  
  categories: true
```

Readers can filter your posts by category automatically — no extra work needed.

Customizing: About Page

`about.qmd` supports beautiful built-in templates:

```
---
title: "Maithreyi Gopalan"
about:
  template: jolla          # trestles, solana, marquee, broadside
  image: profile.jpg
  links:
    - icon: github
      text: GitHub
      href: https://github.com/maithgopalan
    - icon: mortarboard-fill
      text: Google Scholar
      href: https://scholar.google.com/...
    - icon: envelope
      text: Email
      href: mailto:you@uoregon.edu
---
```

I am an Associate Professor of Educational Policy and Leadership

Customizing: Custom CSS

Edit `styles.css` in your project root:

```
/* Change navbar color */
.navbar {
  background-color: #2c3e50 !important;
}

/* Style post titles */
h1.title {
  font-family: 'Source Serif Pro', serif;
  color: #2c3e50;
}

/* Banner on listing page */
.quarto-title-banner {
  background: linear-gradient(135deg, #2c3e50, #3498db);
  color: white;
}
```

Import Google Fonts at the very top of `styles.css`:    30 / 48

More Useful Features

Search — enabled by default, searches all post content:

```
website:  
  search: true
```

Comments — add giscus (GitHub Discussions-based):

```
comments:  
  giscus:  
    repo: YOUR_USERNAME/my-data-blog
```

Google Analytics:

```
website:  
  google-analytics: "G-XXXXXXXXXX"
```

Favicon:

```
website:
```

Troubleshooting Common Issues

Site not updating on GitHub Pages? → Check that `docs/` is committed and `output-dir: docs` is set in `_quarto.yml`

Posts not showing on the listing page? → Make sure posts are inside `posts/` and `draft: false`

Render errors with R code? → Run `quarto render` in Terminal to see full error messages

Fonts not loading? → The `@import` must be the very first line of `styles.css`

GitHub Actions failing? → All R packages used in posts need to be in your `renv.lock` or `DESCRIPTION`

Quick Reference: Key Files

File	Purpose
_quarto.yml	Site-wide settings, theme, navbar
index.qmd	Blog listing page configuration
about.qmd	About/bio page
styles.css	Custom CSS overrides
posts/*/index.qmd	Individual blog posts
_freeze/	Cached computation (commit this!)
docs/	Built site output (commit this!)
.github/workflows/	CI/CD for auto-publishing



You now
have a live
Quarto
blog!

[quarto.org/docs/websites/website-
blog.html](https://quarto.org/docs/websites/website-blog.html)

Data viz in the wild

America and Rachel

Cheyne and Ramtin on Deck for next
week

Deploying Shiny Apps

What is a Shiny App?

- Shiny turns R code into interactive web applications — no web development experience needed
- Typical use cases: interactive data explorers, dashboards, teaching tools, policy visualizations
- Unlike a static blog, Shiny apps require a **live R process** running on a server — so deployment is a different problem

```
install.packages("shiny")  
  
# Minimal app structure:  
# app.R (or ui.R + server.R)
```

Three Deployment Options

Option	Best for	Cost	Effort
shinyapps.io	Sharing publicly, quick deploy	Free tier available	Very low
Posit Connect	Institutional/team use	Paid (UO may have license)	Low
Shinylive	Fully static, no server	Free	Medium

For most course purposes: **shinyapps.io** is the fastest path to a live, shareable app.

Option 1: shinyapps.io

The simplest way to deploy a Shiny app publicly.

Step 1 — Create a free account

Go to shinyapps.io and sign up

Step 2 — Install rsconnect

```
install.packages("rsconnect")
```

Option 1: shinyapps.io

Step 3 — Authorize your account

In RStudio: Tools → Global Options → Publishing → Connect

Or run:

```
rsconnect::setAccountInfo(  
  name    = "YOUR_ACCOUNT_NAME",  
  token   = "YOUR_TOKEN",          # from shinyapps.io dashboard  
  secret  = "YOUR_SECRET"  
)
```


Option 1: shinyapps.io (continued)

Step 4 — Deploy with one click

With your `app.R` open in RStudio, click the **Publish** button (blue icon, top right of editor)

Or deploy from the console:

```
rsconnect::deployApp() # deploys from current working directory
rsconnect::deployApp("path/to/myapp") # or specify a path
```

Your app gets a URL like:

```
https://YOUR_ACCOUNT.shinyapps.io/myapp/
```

Share it — anyone can open it in a browser, no R needed!

Free tier limits: 5 apps, 25 active hours/month. Enough for

Structuring Your App for Deployment

Keep all files your app needs in **one folder**:

```
myapp/
├── app.R          # ui and server in one file (recommended)
├── data/
│   └── clean.csv  # any data files
└── www/
    └── custom.css # optional: images, CSS, JS
```

Common deployment errors:

- File paths — always use relative paths, never `setwd()` or absolute paths
- Missing packages — every package used in `app.R` must be installed on the server
- Data files — must be inside the app folder, not referenced from elsewhere on your computer

Declaring Package Dependencies

shinyapps.io auto-detects packages from `library()` calls, but it's safer to be explicit.

Option A — renv (recommended for serious projects)

```
install.packages("renv")  
renv::init()           # creates renv.lock with all package versions
```

shinyapps.io will read `renv.lock` and install exact versions automatically.

Option B — DESCRIPTION file (lightweight)

Create a `DESCRIPTION` file in your app folder:

Package: myapp

Imports:

Option 2: Posit Connect

If your institution has a **Posit Connect** license (check with your IT/library), this is the best option for:

- Apps that need to stay private or require login
- Scheduled reports and automated workflows
- Sharing within a team or department

Deployment is identical to shinyapps.io — same `rsconnect::deployApp()` command, just pointed at a different server.

```
rsconnect::addServer(  
  url = "https://connect.YOUR_INSTITUTION.edu/",  
  name = "uoregon-connect"  
)  
rsconnect::deployApp(server = "uoregon-connect")
```

Option 3: Shinylive (No Server Needed!)

Shinylive runs Shiny apps entirely in the browser using WebAssembly — **no R server required**.

This means you can host on **GitHub Pages**, just like your Quarto blog.

```
install.packages("shinylive")  
  
# Export app to a static folder  
shinylive::export("myapp/", "site/")  
  
# Then deploy site/ to GitHub Pages
```

Embedding Shinylive in a Quarto Post

You can embed a live Shiny app directly in a [.qmd](#) blog post:

```
```{shinylive-r}
#| standalone: true
#| viewerHeight: 500

library(shiny)
library(ggplot2)

ui <- fluidPage(
 sliderInput("n", "Sample size", 10, 500, 100),
 plotOutput("hist")
)

server <- function(input, output) {
 output$hist <- renderPlot({
 ggplot(data.frame(x = rnorm(input$n)), aes(x)) +
 geom_histogram(bins = 30)
 })
}
```

# Deployment Comparison: Which Should You Use?

---

For this course / quick sharing → shinyapps.io free tier

For embedding interactivity in your Quarto blog → Shinylive

For institutional or team work → Posit Connect

For a polished public portfolio app → shinyapps.io paid, or self-hosted on a cloud VM (Digital Ocean, AWS, etc.)

## Resources

- shinyapps.io docs: [docs.posit.co/shinyapps.io](https://docs.posit.co/shinyapps.io)
- Shinylive: [posit.github.io/rschinylive/](https://posit.github.io/rschinylive/)

# Next time

---

Tables, Geospatial  
visualizations